Mississippi Valley State University

# CS 341 — Discrete Structures

## Lecture 1: What Is Discrete Mathematics?

*Statements, Truth Values, and Precise Reasoning*

Name: _____ Date: _____

**How to use these notes**

Definitions are printed for you — you do not need to copy them. Everything else has space to write. Fill in the blanks *during* class; the parts you write are the parts you will remember. Bring this page back on Wednesday.

## By the end of today you should be able to

1. Say what makes mathematics *discrete* rather than *continuous*.
2. Name several places discrete mathematics shows up in computer science.
3. Decide whether a sentence is a mathematical statement.
4. Give the truth value of a statement when you have enough information.
5. Explain why precise language matters in mathematics *and* in code.

## Warm-up: counting possibilities

A password is exactly four characters long. Each character is either `0` or `1`. How many different passwords are possible?

Your work

**Generalize.** If the password has $n$ binary characters, the number of possibilities is _____.

**Now make it realistic.** Suppose a password is 8 characters, each chosen from the 94 printable ASCII characters. The number of possibilities is _____, which is roughly _____.

**Two ideas to carry away**

**(1)** We just reasoned about *distinct, separate possibilities* — not about a quantity sliding smoothly from one value to the next. That is discrete mathematics. **(2)** "Discrete" does not mean "small." A discrete system can have an astronomical number of configurations. Counting them correctly is exactly why this subject matters to security.

## 1. Discrete vs. continuous

| Continuous mathematics | Discrete mathematics |
|---|---|
| Values vary smoothly. | Values are separate and countable. |
| Usually modeled with real numbers. | Usually modeled with integers and finite structures. |
| Calculus is the central tool. | Logic, counting, graphs, and proofs are the central tools. |
| Example: elapsed time. | Example: number of steps an algorithm takes. |

**Your turn.** Mark each one **D** (discrete) or **C** (continuous).

| Quantity | D / C | Quantity | D / C |
|---|---|---|---|
| Speed of a moving car | | Bits in a file | |
| Students enrolled in CS 341 | | Water level in a tank | |
| Records in a database | | Temperature over the day | |
| Possible passwords | | States of a running program | |

## 2. Where this shows up in computer science

| | |
|---|---|
| **Logic** | Conditions: `if`, `&&`, `\|\|`, `!` |
| **Sets** | Databases, collections, data organization |
| **Functions** | Mappings, transformations, table relationships |
| **Proofs** | Showing an algorithm *always* works, not just on your test cases |
| **Counting** | How many configurations, keys, or possibilities exist |
| **Graphs** | Networks, maps, dependencies, social connections |
| **Trees** | File systems, search, compilers, decision processes |
| **State machines** | Program states, protocols, digital systems |

**The point of the course**

You already write `&&`, `||`, and `!` in CS 204. This course makes them precise enough to *prove things with*. Programming tells a computer what to do. Discrete mathematics lets you specify exactly what should happen and justify that your method is correct.

## 3. Statements

**Definition — statement (also called a proposition)**

A **statement** is a declarative sentence that is either **true** or **false**, but **not both**.

The **truth value** of a statement is either true (**T**) or false (**F**).

**We will work through these together.**

| Sentence | Statement? | Why |
|---|---|---|
| MVSU is in Mississippi. | | |
| $7 + 5 = 13$ | | |
| Is 17 a prime number? | | |
| Close the classroom door. | | |
| This class is interesting. | | |

**Notation — and a warning about symbols**

We name statements with capital letters:

$$P: \quad \text{“17 is a prime number.”}$$

The truth value of $P$ is _____.

Our textbook (Chartrand & Zhang) writes the negation of $P$ as $\sim P$. Many computer science books write $\neg P$, and code writes `!P` or `not P`. **All three mean the same thing**, and all three are accepted on your work in this course.

## 4. Open sentences

**Definition — open sentence**

An **open sentence** contains one or more variables. It is *not* a statement on its own, but it becomes one once the variables are given values (or are quantified).

Let $P(x): \ x + 2 = 7$.

- If $x = 5$, then $P(5)$ is _____.
- If $x = 3$, then $P(3)$ is _____.
- On its own, $x + 2 = 7$ is _____ — it has no truth value yet.

**A trap to avoid all semester**

A sentence is a statement because it *has* a definite truth value — **not** because you happen to know that value, and **not** because you could conveniently check it. "There are an even number of grains of sand on Earth" is a perfectly good statement. Nobody knows which one it is.

## 5. Practice: classify each sentence

Use one of: **true statement** • **false statement** • **not a statement** • **open sentence**

| | Sentence | Classification | Reason |
|---|---|---|---|
| 1. | $2^5 = 32$ | | |
| 2. | 10 is an odd integer. | | |
| 3. | What time does class end? | | |
| 4. | Submit the assignment through Canvas. | | |
| 5. | $x + 2 = 7$ | | |
| 6. | Every computer program eventually terminates. | | |
| 7. | There are 25 students in this room. | | |

**Look closely at #6 and #7**

**#6** is refuted by producing a single example that breaks it. That move is called a *counterexample*, and it becomes a formal proof technique in Chapter 3.
**#7** is a statement even before anyone counts. Compare it with #5: $x + 2 = 7$ has no truth value

*at all* until $x$ is fixed. That is a real difference, not a technicality.

## 6. Small words carry the logic

Read this rule carefully:

A user is granted access if the password is correct **and** the account is not locked.

1. Password correct, account not locked. Access? _____
2. Password correct, account locked. Access? _____
3. Password wrong, account not locked. Access? _____
4. Which single word controls all of this? _____

Now write the same rule as code:

```
if password_correct and not account_locked:
    grant_access()
```

**What changes if we replace `and` with `or`?**

**Central message**

Small words — *and, or, not, if, only if, all, some* — decide the meaning of a mathematical statement and the behavior of a program. Most of this course is about removing ambiguity from reasoning.

## 7. Looking ahead: negation

Let $P$ : "The program terminates."

Then $\sim P$ : _____

| $P$ | $\sim P$ |
|---|---|
| T | |
| F | |

A negation must assert the *logical opposite*. It is not the same as merely disagreeing, and it is not the same as saying the opposite is extreme. The negation of "the program terminates" is not "the program crashes."

## Symbols to know (keep this — you will use it all semester)

| Symbol | Name | Read as | In code |
|---|---|---|---|
| $\sim P \quad (\neg P)$ | negation | not $P$ | `!p  not p` |
| $P \wedge Q$ | conjunction | $P$ and $Q$ | `p && q  p and q` |
| $P \vee Q$ | disjunction | $P$ or $Q$ (inclusive) | `p \|\| q  p or q` |

Wednesday we define $\wedge$ and $\vee$ properly and build truth tables for them.

## Exit ticket

1. Is "$n$ is an even integer" a statement? Explain briefly.

2. Write one statement of your own that relates to computing, and give its truth value.

3. In one sentence: why is discrete mathematics useful to a computer scientist?

## Questions I still have

## Before Wednesday

**Read** (Chartrand & Zhang):

- Chapter 0, "What Is Discrete Mathematics?" — pp. 1–6
- Section 1.1, "Statements" — pp. 7–12
- Preview Section 1.2 through the negation subsection — pp. 13–15

**Do not have the book yet?** The same material is covered free in Chapter 3 of *Mathematics for Computer Science* (Lehman, Leighton, and Meyer), linked on Canvas.

**Bring to class Wednesday (ungraded, but come prepared)**

Three sentences of your own: one **true statement**, one **false statement**, and one sentence that is **not a statement**. At least one must relate to computing.